



Gruppo SkyNet
skynetunigroup@gmail.com

Manuale Utente

Code Guardian

Versione	1.0.0
Uso	<i>Utenti finali di Code Guardian</i>
Redattori	<i>Samuele Bertin</i>
Verifica	<i>Dario Pirrone, Andrea Fistarol</i>
Approvazione	<i>Dario Pirrone, Marco Barbiero</i>

Versioni del documento

Versione	Data	Descrizione	Autore	Verificatore
1.0.0	2026/09/23	Approvazione per la revisione PB	Marco Barbiero	-
0.5.0	2026/09/20	Allineamento del Manuale alle modalità operative effettive. Controllo ortografico e di presentazione del documento.	Andrea Fistarol	Edoardo Granziero
0.4.0	2026/09/06	Risoluzione dei problemi ed errori, Glossario.	Samuele Bertin	Dario Pirrone
0.3.0	2026/09/05	Funzionalità AI.	Samuele Bertin	Dario Pirrone
0.2.0	2026/09/04	Installazione e Avvio, Guida all'interfaccia.	Samuele Bertin	Dario Pirrone
0.1.0	2026/09/03	Prima stesura del Manuale Utente: Introduzione.	Samuele Bertin	Dario Pirrone

Contents

1	Introduzione	3
1.1	Cos'è Code Guardian	3
1.2	A chi è rivolto questo manuale	3
1.3	I tre agenti in breve	3
1.4	Ruoli operativi	3
2	Installazione e Avvio	4
2.1	Requisiti	4
2.2	Creazione dell'account	4
2.3	Accesso (Login)	5
2.4	Configurazione dei servizi esterni (GitHub)	5
2.5	Primo utilizzo: selezione del contesto operativo	5
3	Guida all'interfaccia	6
3.1	Mappa di navigazione	6
3.2	Intestazione applicativa	6
3.3	Servizi connessi — <code>/credentials</code>	7
3.4	Selezione del contesto — <code>/select</code>	7
3.5	Avvio operazioni — <code>/run</code>	7
3.6	Dashboard — <code>/tasks</code>	7
3.7	Storico e dettaglio dei report — <code>/reports, /reports/:id</code>	8
3.8	Convenzioni visive	9
4	Funzionalità AI	10
4.1	Come funzionano gli agenti, in generale	10
4.2	Agente Docs	10
4.3	Agente Security	10
4.4	Agente Changelog	11
4.5	Collegamento alla Pull Request	11
4.6	Esportazione dei report in PDF	11
5	Risoluzione problemi ed errori	12
5.1	Due famiglie di errore	12
5.2	Tabella degli errori più comuni	13
5.3	Credenziale GitHub non più valida	14
5.4	Limite di utilizzo raggiunto	14
5.5	Altri casi frequenti	14
6	Glossario	15

1 Introduzione

1.1 Cos'è Code Guardian

Code Guardian è un'applicazione web che automatizza l'audit, la manutenzione e la documentazione dei repository software ospitati su GitHub. Una volta collegato il proprio repository, l'utente può richiedere a tre agenti intelligenti, basati su modelli linguistici di grandi dimensioni (LLM), di analizzare il codice e produrre in autonomia:

- documentazione tecnica aggiornata (README, commenti inline, documentazione degli Endpoint);
- una scansione delle vulnerabilità di sicurezza più comuni;
- un changelog leggibile a partire dalle User Story completate in uno Sprint.

Ogni analisi produce un **report** strutturato, consultabile nell'interfaccia e scaricabile in formato PDF. Quando un agente propone una modifica ai file del progetto (ad esempio un nuovo README), Code Guardian apre automaticamente una **Pull Request** su GitHub: nessuna modifica viene mai scritta direttamente sul repository senza che sia l'utente, tramite il normale flusso di revisione di GitHub, ad approvarla.

1.2 A chi è rivolto questo manuale

Questo manuale è rivolto a chiunque utilizzi Code Guardian come utente finale: sviluppatori, security auditor/team lead e project manager. Non tratta argomenti di infrastruttura o di amministrazione del sistema (hosting, configurazione dei server, gestione del database), che restano di competenza del team che opera e mantiene l'applicazione.

1.3 I tre agenti in breve

- **Agente Docs** — genera e aggiorna la documentazione del progetto: il file `README.md`, i commenti di codice (JSDoc/docstring) e un documento `API.md` con l'elenco degli Endpoint. Propone sempre le modifiche tramite Pull Request.
- **Agente Security** — analizza il codice alla ricerca di vulnerabilità riconducibili alla OWASP Top 10 e verifica la conformità del progetto alle regole definite nel file `POLICY.md`. Segnala i problemi riscontrati ma non modifica mai il codice: la correzione resta un'attività umana.
- **Agente Changelog** — legge le User Story completate in uno Sprint da GitHub Issues e genera un changelog tecnico, orientato allo sviluppo, e, su richiesta, un changelog di business più leggibile per un pubblico non tecnico.

1.4 Ruoli operativi

Ogni utente sceglie un ruolo in fase di registrazione, che resta fisso per tutta la vita dell'account e determina quali operazioni sono disponibili nella schermata di avvio. La Tabella 1 riassume la mappatura.

Tutti i ruoli condividono inoltre le funzionalità trasversali dell'applicazione: selezione del repository, configurazione delle credenziali, monitoraggio delle operazioni in corso e consultazione dello storico dei report. Le operazioni non permesse al proprio ruolo non vengono mostrate come disabilitate: semplicemente non compaiono nell'elenco.

Ruolo	Operazioni disponibili	Agente
Sviluppatore	Generazione/aggiornamento README, documentazione inline (JSDoc), documentazione API	Docs
Security Auditor / Team Lead	Scansione vulnerabilità OWASP Top 10, verifica conformità Policy-as-code	Security
Project Manager	Changelog di business	Changelog
Project Manager e Sviluppatore	Changelog tecnico	Changelog

Table 1: Operazioni disponibili per ciascun ruolo operativo.

2 Installazione e Avvio

Code Guardian è un'applicazione interamente web-based: non richiede alcuna installazione sul proprio computer. È sufficiente un browser aggiornato per accedervi.

2.1 Requisiti

- Un browser desktop recente e aggiornato: Google Chrome, Mozilla Firefox oppure Microsoft Edge (versione 145 o superiore).
- Un account GitHub con un Personal Access Token (PAT) con permesso `repo`, per collegare i propri repository (si veda la sezione 2.4).
- L'indirizzo (URL) dell'ambiente Code Guardian fornito dal proprio team o amministratore.

Attenzione — Il token di sessione dell'utente non viene mai salvato nella memoria persistente del browser. Questo significa che, ricaricando completamente la pagina, è necessario effettuare di nuovo l'accesso: si tratta di una scelta di sicurezza, non di un malfunzionamento.

2.2 Creazione dell'account

Al primo utilizzo occorre creare un account dalla pagina `/register`:

1. inserire nome e cognome;
2. inserire un indirizzo email valido (verrà validato mentre si compila il campo);
3. scegliere una password di almeno 8 caratteri, con almeno una lettera e una cifra;
4. selezionare il proprio ruolo operativo iniziale: Sviluppatore, Security Auditor / Team Lead oppure Project Manager (si veda la sezione 3 per le differenze);
5. confermare con il pulsante **Crea account**.

Se l'indirizzo email è già registrato, compare un messaggio non bloccante con un collegamento diretto alla pagina di accesso. La registrazione non effettua l'accesso automatico: una volta creato l'account occorre accedere separatamente da `/login`.

Nota —Il ruolo scelto in fase di registrazione resta fisso per tutta la vita dell'account: non esiste, in questa versione dell'applicazione, una funzione per cambiarlo in un secondo momento.

2.3 Accesso (Login)

Dalla pagina `/login`, inserire l'indirizzo email e la password scelti in fase di registrazione e confermare con **Accedi**. In caso di credenziali non corrette, viene mostrato un unico messaggio generico (per non rivelare se l'errore riguarda l'email o la password). La sessione resta attiva per 8 ore o fino al ricaricamento completo della pagina.

Dopo l'accesso, l'applicazione reindirizza automaticamente l'utente:

- alla pagina `/credentials`, se non è ancora stato configurato alcun servizio esterno;
- alla pagina `/select`, se una credenziale GitHub è già configurata e valida.

2.4 Configurazione dei servizi esterni (GitHub)

Nessuna operazione degli agenti è raggiungibile senza aver prima collegato un account GitHub. Sulla pagina `/credentials`:

1. inserire il proprio Personal Access Token GitHub con permesso **repo** nel campo dedicato;
2. confermare con **Connetti**: il sistema esegue prima un controllo di formato, poi una chiamata di verifica a GitHub; la credenziale viene salvata solo se entrambi i controlli hanno esito positivo;
3. una volta connesso, il servizio mostra l'etichetta **Connesso** con la data dell'ultima verifica riuscita.

Lo stesso token copre sia la lettura del codice sia l'accesso alle GitHub Issues, usate dall'Agente Changelog come sistema di gestione dei task: non è quindi necessario configurare due credenziali separate.

Nota —In qualunque momento è possibile premere **Verifica di nuovo** per controllare che il token sia ancora valido, oppure **Rimuovi** per revocarlo localmente (l'operazione non revoca il token anche su GitHub: per farlo occorre agire direttamente sul proprio account GitHub).

2.5 Primo utilizzo: selezione del contesto operativo

Completata la configurazione delle credenziali, si accede alla pagina `/select` per definire su cosa dovranno lavorare gli agenti (si veda la sezione 3.4 per il dettaglio dei campi). Una volta salvato, il contesto resta valido per tutte le operazioni avviate nella stessa sessione di lavoro; agli accessi successivi, un utente già configurato salta direttamente alla pagina `/run` o a `/select` se desidera cambiare repository.

3 Guida all'interfaccia

3.1 Mappa di navigazione

La Figura 1 riassume le pagine principali dell'applicazione e i percorsi che un utente segue normalmente per andare dall'accesso all'avvio di un'operazione e alla consultazione del relativo report.

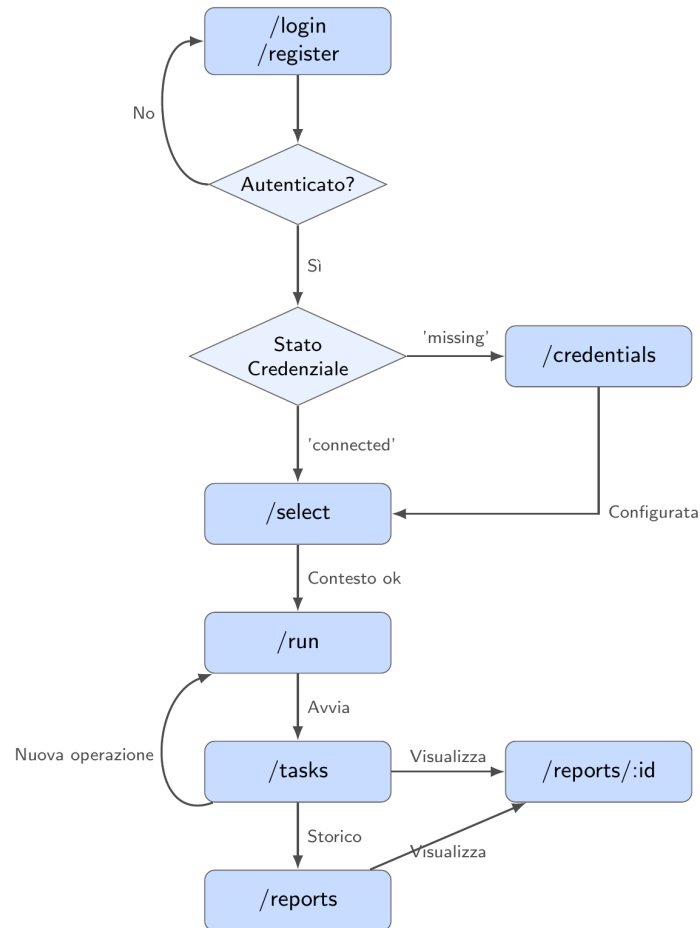


Figure 1: Mappa di navigazione principale dell'applicazione.

3.2 Intestazione applicativa

Un'intestazione persistente compare su tutte le pagine successive all'accesso e mostra:

- il nome dell'utente autenticato;
- un'etichetta con il ruolo operativo assegnato;
- un banner di avviso, quando la credenziale GitHub non è più valida (si veda la sezione 5.3), con un collegamento diretto a `/credentials`;
- il comando **Esci**, che chiude la sessione corrente.

3.3 Servizi connessi — /credentials

Elenca i servizi esterni collegabili (nell'attuale versione, solo GitHub) con il relativo stato: non configurato, connesso oppure non più valido. È la pagina descritta in dettaglio nella sezione 2.4.

3.4 Selezione del contesto — /select

Questa pagina definisce *su cosa* lavoreranno gli agenti. È divisa in due aree.

Repository e riferimento.

- **URL del repository** — l'indirizzo del repository GitHub da analizzare.
- **Branch** — il nome del ramo di riferimento; una volta validato l'URL, il campo propone un autocompletamento con i branch disponibili.
- **Hash del commit** (opzionale) — per ancorare l'analisi a un commit specifico anziché all'ultimo dello stato attuale del branch.

Ambito di analisi. Una volta validato il repository, si sceglie *quanto* del repository analizzare:

- **Intero repository** — comando rapido che seleziona tutto il contenuto;
- **File o cartelle specifici** — si deve poi inserire nella casella di testo sottostante l'indirizzo o gli indirizzi dei file o delle cartelle da esaminare.

Nota — Solo i file scritti in un linguaggio supportato (TypeScript, JavaScript o Python) vengono effettivamente considerati dagli agenti. Se il repository contiene altri linguaggi, compare un avviso non bloccante: l'analisi prosegue comunque sui file supportati.

3.5 Avvio operazioni — /run

Mostra l'elenco delle operazioni disponibili per il proprio ruolo (si veda la Tabella 1), ciascuna con nome e breve descrizione. È possibile selezionare più operazioni contemporaneamente e avviarle in un'unica conferma con il pulsante **Avvia**: ciascuna diventerà un elemento indipendente nella Dashboard. Il pulsante resta disabilitato finché non è selezionata almeno un'operazione.

Nota — L'operazione *Changelog di business* non richiede di selezionare anche il *Changelog tecnico*: la generazione del changelog tecnico è inclusa automaticamente come primo passo dell'operazione di business (si veda la sezione 4.4).

3.6 Dashboard — /tasks

La Dashboard elenca le operazioni avviate nella sessione corrente e ne mostra l'avanzamento in tempo reale, senza necessità di ricaricare la pagina. Per ciascuna operazione sono visibili il nome, un indicatore di stato con barra di avanzamento, e i comandi disponibili.

Ogni operazione attraversa gli stati mostrati in Figura 2:

- **In attesa** — l'operazione è stata accodata mentre attende una risorsa disponibile;

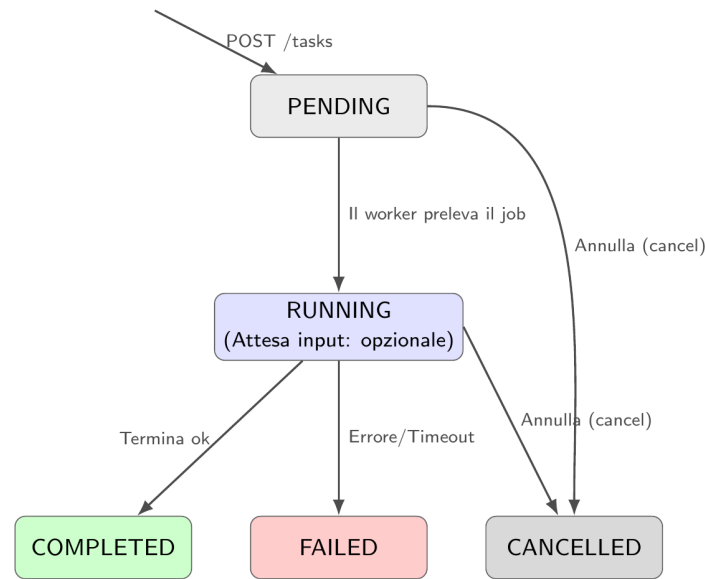


Figure 2: Ciclo di vita di un'operazione (Task) mostrata in Dashboard.

- **In elaborazione** — l'agente sta analizzando il contenuto selezionato (è in questo stato che, per l'Agente Changelog, possono comparire richieste di input, si veda la sezione 4.4);
- **Completato** — il report è pronto e consultabile;
- **Fallito** — l'operazione si è interrotta per un errore (si veda la sezione 5);
- **Annullato** — l'operazione è stata interrotta dall'utente con il comando **Annulla**, disponibile finché l'operazione è in attesa o in elaborazione.

Nota — Le operazioni avviate insieme vengono eseguite una alla volta, non in parallelo: se si avviano più operazioni contemporaneamente, alcune resteranno nello stato In attesa finché l'agente non si libera dall'elaborazione precedente. Il fallimento di un'operazione, in ogni caso, non blocca né invalida quelle in coda dopo di essa.

Il comando **Visualizza report** è disponibile per le operazioni completate o fallite. Il fallimento di un'operazione è sempre isolato al singolo elemento della lista: le altre operazioni del batch proseguono senza interruzioni.

3.7 Storico e dettaglio dei report — /reports, /reports/:id

La pagina **/reports** elenca tutti i report generati dall'utente, con identificativo, titolo descrittivo (nome dell'operazione, repository e branch analizzati) e data/ora di generazione. Selezionando un elemento si apre la pagina di dettaglio **/reports/:id**, raggiungibile anche direttamente dalla Dashboard subito dopo il completamento di un'operazione.

La pagina di dettaglio mostra, nell'intestazione, repository, riferimento, ambito analizzato e tempo di esecuzione; nel corpo, il contenuto del report nel formato più adatto al tipo di operazione (testo Markdown, elenco di riscontri, collegamento alla Pull Request — si veda la sezione 4). Da qui è sempre disponibile il pulsante **Esporta PDF**.

3.8 Convenzioni visive

- **Colori di stato** — ogni indicatore di stato (Attesa, Elaborazione, Completato, Fallito, Annullato) ha un colore coerente in tutta l'applicazione, sempre accompagnato da un'etichetta testuale.
- **Colori di gravità** — i riscontri dell'Agente Security e gli avvisi di complessità dell'Agente Docs usano gli stessi cinque livelli: Critica, Alta, Media, Bassa, Informativa.
- **Blocchi di codice** — gli snippet correttivi e i riferimenti a percorsi di file/righe sono mostrati con un font a spaziatura fissa, distinto dal testo dell'interfaccia.
- **Stati delle richieste** — ogni azione che comporta una chiamata di rete mostra un indicatore di caricamento durante l'attesa e disabilita il comando che l'ha originata, per evitare invii duplicati.

4 Funzionalità AI

4.1 Come funzionano gli agenti, in generale

Ogni agente segue lo stesso schema di funzionamento:

1. legge il codice (e, per il Changelog, le GitHub Issues) esclusivamente *in lettura*, tramite l'accesso sicuro predisposto dall'applicazione — gli agenti non ricevono mai direttamente il token GitHub dell'utente;
2. elabora il contenuto con un modello linguistico (LLM), seguendo un modello di prompt specifico per l'operazione richiesta;
3. restituisce un **report** strutturato, che l'applicazione mostra nell'interfaccia e rende disponibile per l'esportazione in PDF.

Nota — Nessun agente scrive mai direttamente sul repository. Quando un'operazione prevede una modifica ai file (l'Agente Docs), l'agente produce soltanto una proposta (un diff); è l'applicazione stessa, non l'agente, ad aprire in automatico la Pull Request corrispondente al termine dell'elaborazione.

4.2 Agente Docs

Copre tre operazioni, tutte riservate al ruolo Sviluppatore e tutte concluse con l'apertura automatica di una Pull Request:

- **Generazione/aggiornamento README** — produce o aggiorna il file `README.md` del progetto;
- **Documentazione inline** — analizza il codice sorgente e propone commenti nella convenzione corretta (JSDoc per TypeScript/JavaScript, docstring per Python), sia per le unità di codice prive di documentazione sia per quelle la cui documentazione esistente non è più coerente con il codice;
- **Documentazione API** — estrae le informazioni sugli Endpoint del progetto e genera un documento `API.md`.

Il report mostra come elemento principale il collegamento alla Pull Request aperta su GitHub: la revisione e l'eventuale approvazione della modifica proposta avvengono interamente su GitHub, con i normali strumenti di revisione (Code Guardian non prevede un passaggio di approvazione interno). Per la documentazione inline e API, se una porzione di codice è troppo complessa per essere documentata in modo affidabile, il report include un avviso dedicato (a gravità informativa) invece di forzare un commento potenzialmente scorretto.

4.3 Agente Security

Copre due operazioni, riservate al ruolo Security Auditor / Team Lead:

- **Scansione vulnerabilità OWASP Top 10** — analizza il codice alla ricerca di pattern riconducibili alle vulnerabilità più comuni (ad esempio SQL injection, cross-site scripting, segreti in chiaro, crittografia insicura, controllo accessi incompleto);
- **Verifica conformità Policy-as-code** — confronta il progetto con le regole di sviluppo sicuro definite nel file `POLICY.md` presente nel repository.

Il report elenca i riscontri come lista filtrabile per gravità; per ciascun riscontro sono indicati categoria, gravità, file e riga (quando disponibili) e un suggerimento di correzione, sotto forma di uno snippet di codice quando il problema è sufficientemente localizzato, oppure di una descrizione testuale in caso contrario.

Attenzione — L'Agente Security individua e documenta i problemi, ma non li corregge né apre alcuna Pull Request: la verifica e la correzione restano un'attività umana, a cura del Security Auditor.

4.4 Agente Changelog

Copre due operazioni: il **Changelog tecnico** (Project Manager e Sviluppatore) e il **Changelog di business** (solo Project Manager). È l'agente con il flusso più interattivo, perché richiede alcune informazioni e conferme direttamente durante l'elaborazione, senza che l'operazione venga interrotta.

Come mostrato in Figura 3, durante l'esecuzione possono comparire fino a tre richieste, direttamente come area interattiva sull'elemento della Dashboard:

1. **Richiesta dello Sprint ID** — l'utente inserisce l'identificativo dello Sprint da cui estrarre le User Story completate, oppure annulla l'operazione;
2. **Metadati insufficienti** — se alcune User Story hanno una descrizione troppo povera per essere incluse in modo affidabile, l'utente sceglie se **procedere escludendole** oppure **annullare** l'intera operazione; vengono comunque riportate nel report come elenco delle esclusioni;
3. **Conferma del changelog di business** — solo per l'operazione di business: una volta generato il changelog tecnico intermedio, l'utente lo consulta in anteprima e sceglie se **accettare e generare il changelog di business** oppure **annullare**. Un rifiuto in questo passo annulla l'intera operazione: chi desidera solo il changelog tecnico può avviare direttamente quella singola operazione.

Il changelog prodotto è un testo in formato Markdown, arricchito con i collegamenti ipertestuali ai ticket di origine su GitHub. Il changelog di business viene inoltre verificato dall'agente stesso per garantire un livello minimo di leggibilità prima di essere restituito.

4.5 Collegamento alla Pull Request

Per tutte le operazioni dell'Agente Docs, il report include un collegamento diretto alla Pull Request aperta automaticamente su GitHub al termine dell'elaborazione, insieme a un'anteprima del diff proposto (consultabile ma secondaria rispetto al collegamento). Nessun'altra azione è richiesta in Code Guardian: l'accettazione, il rifiuto o la richiesta di modifiche avvengono normalmente su GitHub.

4.6 Esportazione dei report in PDF

Da qualunque pagina di dettaglio di un report è disponibile il pulsante **Esporta PDF**, che genera e scarica un documento PDF con lo stesso contenuto mostrato a schermo. Il file viene generato al momento della prima richiesta e riutilizzato per le richieste successive, poiché il contenuto di un report non cambia nel tempo. Un eventuale fallimento dell'esportazione non altera lo stato del report né il contenuto già visualizzato: viene mostrato solo un messaggio d'errore temporaneo (si veda la sezione 5).

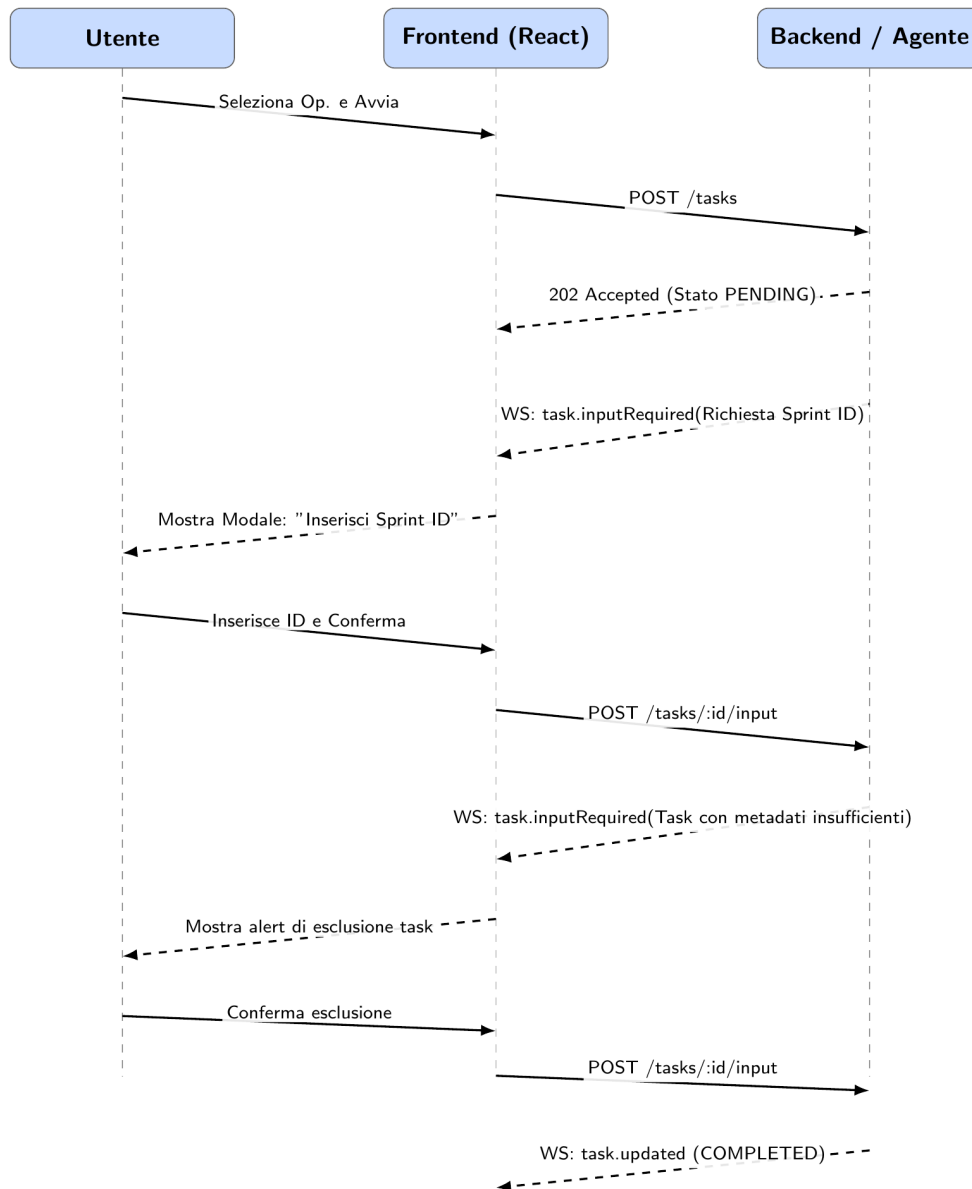


Figure 3: Sequenza di interazione tra utente e Agente Changelog durante l’elaborazione.

5 Risoluzione problemi ed errori

5.1 Due famiglie di errore

Code Guardian distingue due tipi di errore, gestiti in modo diverso dall’interfaccia:

- **Errori di validazione e configurazione** — emergono immediatamente mentre si compila un modulo o si conferma una selezione (ad esempio un formato email non valido, un token GitHub mal formato). Il messaggio compare direttamente sotto il campo interessato e i dati già inseriti non vengono mai persi.
- **Errori di esecuzione** — emergono a distanza di tempo dall’avvio di un’operazione, mentre l’agente sta lavorando, e vengono notificati in tempo reale. Solo l’operazione interessata passa allo stato **Fallito**: le altre proseguono senza interruzioni. Aprendo il report di un’operazione fallita, il contenuto normale viene sostituito da un riquadro che indica il

tipo di errore, la fase in cui si è verificato e, quando disponibile, un'azione secondaria da intraprendere.

5.2 Tabella degli errori più comuni

La Tabella 2 elenca le cause più comuni di fallimento di un'operazione, il messaggio che ci si può aspettare di vedere e l'azione consigliata.

Causa	Messaggio tipico	Cosa fare
Timeout del modello AI (TIMEOUT)	Il modello AI ha impiegato troppo tempo a rispondere.	Riprova: per rilanciare la sola operazione fallita.
Output del modello non interpretabile (PARSING)	L'output generato dall'AI non è nel formato atteso.	Riprova: nella maggior parte dei casi un nuovo tentativo va a buon fine.
Errore generico di sistema (UPSTREAM)	Impossibile completare l'operazione per un errore interno di sistema.	Riprova; se il problema persiste, contattare chi amministra l'applicazione.
Ambito di analisi troppo esteso (CONTEXT_TOO_LARGE)	Il contesto analizzato eccede la capacità di lettura del modello.	Tornare a <code>/select</code> e ridurre l'ambito (selezionare meno file o cartelle).
Risorsa mancante nel repository (CONTEXT_RESOURCE_MISSING)	Risorsa richiesta non trovata (ad es. <code>POLICY.md</code> assente, Sprint ID inesistente).	Verificare che la risorsa richiesta esista nel repository o nel Task Management, quindi riavviare l'operazione.
Risorsa non interpretabile (CONTEXT_RESOURCE_INVALID)	La risorsa richiesta non è interpretabile.	Controllare il formato della risorsa indicata (ad es. il file <code>POLICY.md</code>) e correggerlo prima di riprovare.
Limite di chiamate al modello AI (LLM_RATE_LIMITED)	Limite di utilizzo del modello AI raggiunto; riprovare successivamente.	Attendere qualche minuto prima di riavviare l'operazione.
Leggibilità del changelog insufficiente (READABILITY_TOO_LOW)	Il changelog di business generato non raggiunge la soglia minima di leggibilità richiesta.	Rilanciare l'operazione; se il problema persiste, provare a ridurre l'ambito o lo Sprint selezionato.
Apertura Pull Request non riuscita (PULL_REQUEST_CREATION_FAILED)	Impossibile aprire la Pull Request; verificare i permessi del token.	Controllare i permessi del token in <code>/credentials</code> e riavviare l'operazione.
Limite mensile di operazioni raggiunto (USAGE_LIMIT_EXCEEDED)	Limite di operazioni avviabili raggiunto per il periodo corrente.	Attendere il rinnovo del periodo, oppure contattare chi amministra l'applicazione.
Token GitHub scaduto o revocato (CREDENTIAL_INVALID)	Il token configurato non è più valido; verificalo su "Servizi connessi".	Riconfigurare il token in <code>/credentials</code> (si veda la sezione 5.3).

Table 2: Errori più comuni, messaggio mostrato e azione consigliata.

5.3 Credenziale GitHub non più valida

Se durante l'esecuzione di un'operazione il token GitHub configurato risulta scaduto o revocato, l'operazione in corso fallisce con l'errore `CREDENTIAL_INVALID` e l'applicazione mostra un banner persistente nell'intestazione, visibile su tutte le pagine, finché la credenziale non viene sistemata. L'avvio di nuove operazioni resta bloccato nel frattempo, mentre restano sempre consultabili le operazioni e i report già esistenti. Per risolvere:

1. aprire `/credentials` dal collegamento presente nel banner;
2. generare un nuovo Personal Access Token su GitHub (o rinnovare i permessi di quello esistente);
3. inserirlo nel campo del servizio GitHub e confermare: una volta validato, il banner scompare e l'applicazione torna pienamente utilizzabile.

5.4 Limite di utilizzo raggiunto

Per contenere il consumo del modello AI, Code Guardian applica un limite al numero di operazioni avviabili da ciascun utente in un mese solare. Se il limite viene raggiunto, l'avvio di una nuova operazione (o di un intero lotto di operazioni avviate insieme) viene rifiutato prima ancora che un agente venga interpellato: nessuna operazione del lotto parte. Occorre attendere il rinnovo del periodo mensile oppure contattare chi amministra l'applicazione per un eventuale innalzamento del limite.

5.5 Altri casi frequenti

- **Il comando “Esci” mostra un errore** — il logout richiede comunque una chiamata di rete; in caso di problemi di connessione può comparire un errore contestuale. La sessione locale, in ogni caso, viene comunque interrotta al successivo ricaricamento della pagina.
- **L'esportazione PDF fallisce** — compare un messaggio temporaneo nella pagina del report, che resta comunque consultabile a schermo: è sufficiente premere di nuovo **Esporta PDF** per riprovare.
- **Il repository o il branch non vengono trovati** — verificare che l'URL sia corretto e che il token configurato in `/credentials` abbia accesso al repository (per i repository privati, il token deve appartenere a un account con i permessi necessari).

6 Glossario

Agente	Componente basato su un modello linguistico (LLM) che esegue una specifica operazione di analisi o generazione (Docs, Security o Changelog) e restituisce un Report.
Ambito di analisi	La porzione del repository (intero contenuto, singoli file o cartelle) su cui un agente lavora, definita nella pagina <code>/select</code> .
Branch	Un ramo del repository Git; nella selezione del contesto rappresenta il riferimento di base su cui viene ancorata l'analisi.
Changelog di business	Sintesi delle novità di uno Sprint, scritta con un linguaggio accessibile a un pubblico non tecnico.
Changelog tecnico	Sintesi delle novità di uno Sprint orientata a un pubblico di sviluppatori, con collegamenti ai ticket di origine.
Commit	Un singolo salvataggio di modifiche nella cronologia del repository, identificato da un hash univoco; può essere usato come riferimento opzionale per ancorare un'analisi a un punto preciso della cronologia.
Contesto (di analisi)	L'insieme di repository, riferimento (branch/commit) e ambito selezionato, condiviso da tutte le operazioni avviate nella stessa sessione di lavoro.
Credenziale di servizio	Il Personal Access Token GitHub configurato dall'utente, che consente a Code Guardian di leggere codice e Issue per suo conto.
Dashboard	La pagina <code>/tasks</code> , che mostra in tempo reale lo stato delle operazioni avviate.
GitHub Issues	Il sistema di ticket di GitHub, usato in Code Guardian anche come sistema di gestione dei task/User Story letto dall'Agente Changelog.
JWT (token di sessione)	Il token che identifica una sessione utente autenticata; viene inviato automaticamente a ogni richiesta e non viene mai salvato in modo persistente nel browser.
LLM (Large Language Model)	Il modello linguistico di intelligenza artificiale su cui si basa ciascun agente per interpretare il codice e generare l'output.
OWASP Top 10	Elenco delle categorie di vulnerabilità di sicurezza più comuni nelle applicazioni web, usato come riferimento dalla Scansione vulnerabilità dell'Agente Security.
Personal Access Token (PAT)	Il token di accesso personale generato su GitHub, usato come credenziale di servizio.

POLICY.md	File presente nel repository che definisce le regole di sviluppo sicuro del progetto, verificato dall'operazione di conformità Policy-as-code.
Proposal	La modifica proposta da un agente sotto forma di diff (ad esempio un nuovo README), da cui viene generata automaticamente una Pull Request.
Pull Request (PR)	Una richiesta di modifica al repository, aperta automaticamente da Code Guardian al termine di un'operazione dell'Agente Docs; la sua revisione e approvazione avvengono su GitHub.
Repository	Archivio remoto, ospitato su GitHub, contenente il progetto software su cui vengono eseguite le analisi.
Report	Il risultato strutturato di un'operazione, consultabile nell'interfaccia ed esportabile in PDF.
Ruolo operativo	La specializzazione scelta in fase di registrazione (Sviluppatore, Security Auditor / Team Lead o Project Manager), che determina le operazioni disponibili.
Sprint	Un intervallo di lavoro del team, identificato da uno Sprint ID, da cui l'Agente Changelog estrae le User Story completate.
Task	Una singola operazione avviata (ad esempio una scansione di sicurezza), mostrata come elemento indipendente in Dashboard con un proprio stato e un proprio report.
WebSocket	Il canale di comunicazione in tempo reale tramite cui l'interfaccia riceve gli aggiornamenti di stato delle operazioni senza dover ricaricare la pagina.